

Symbolic Verification of the WireGuard Handshake and Its Post-Quantum Migration Cost

Qiyue Zhao

Abstract—We present a verification artifact for the WireGuard handshake in which a single protocol transcription drives two symbolic provers and every number printed in this paper is generated from what those provers wrote. The design buys two things at once: any difference between the verdicts is attributable to the prover rather than to the modelling, and no claim in the text can outlive the run that produced it.

Four results follow. *Firstly*, a controlled pair of models isolates what the pre-shared key of `IKpsk2` contributes: against an attacker granted a discrete-logarithm oracle after the session closes, transport-key secrecy is proved when the pre-shared key is secret and refuted when it is published, with the models otherwise identical. *Secondly*, key-compromise impersonation resistance is located rather than asserted—with the responder’s static key leaked the responder does commit transport keys to a session it attributes to an honest peer, and never accepts traffic on it; the static-ephemeral term *se* holds that interval open. *Moreover*, an arithmetic analysis carries post-quantum parameter sizes past the cryptography and into the network layer, where an in-band ML-KEM-768 initiation reaches 1300 B against a usable IPv6 minimum-MTU payload of 1232 B. *Finally*, the failures are reported alongside the proofs: 3 queries were left undecided, and they are in the same table as the rest.

The numbers make the comparison concrete. Across 18 properties the two provers return 11 proofs, 3 attacks, 1 inconclusive and 3 non-terminating verdicts. ProVerif decides 9 of 12 properties in <1–24 s and exhausts a 300 s budget on the other 3; Tamarin decides all 6, including both key-compromise-impersonation levels, in 2–3 s and 7–18 constraint-solving steps. The in-band initiation grows by a factor of 8.8 at ML-KEM-768 and overshoots the IPv6 minimum-MTU payload by 68 B, so no in-band construction has a guaranteed path—which is why separate-channel designs are architecturally necessary rather than merely convenient.

1. Introduction

WireGuard [1] is a VPN protocol whose handshake instantiates the Noise `IKpsk2` pattern [2]. It sits in the mainline Linux kernel and carries production traffic, so questions about its security properties, and about what migrating it to post-quantum primitives would cost, are engineering questions with deployed consequences.

The protocol has been analysed before: symbolically by Donenfeld and Milner [3], and in the computational model by Lipp, Blanchet and Bhargavan [4]. This work is an independent reconstruction from the specification and claims no new vulnerability. Its contribution lies elsewhere, in two places that prior analyses did not occupy. The first is methodological: the handshake is transcribed *once* and handed to two provers, so that a disagreement between them is evidence about the provers rather than about two people’s modelling habits. The second is that the analysis does not stop at the cryptography. Post-

quantum parameter sizes are followed through to datagram sizes and from there to path MTUs, which is where the migration cost actually becomes visible.

1.1. Contributions

A controlled experiment isolating the pre-shared key. The `psk` of `IKpsk2` is widely described as providing post-quantum protection. Section 4 makes that claim testable with a pair of models identical in every respect except whether the pre-shared key is secret, run against an attacker that gains a discrete-logarithm oracle after the session completes. Transport-key secrecy is proved in one and refuted in the other. Because the models differ in exactly one variable, so does the explanation.

Locating key-compromise impersonation resistance. With the responder’s own static key in the attacker’s hands, we show that the responder *does* commit transport keys for a session it attributes to an honest initiator, and *never* accepts traffic on that session. Forging message 1 needs only Diffie-Hellman terms computable from the leaked scalar; deriving the transport keys additionally needs the static-ephemeral term $se = \text{DH}(e_r, S_i)$, which does not follow. The property therefore lives in the interval between committing key material and accepting traffic—a distinction a single monolithic “session established” event would hide entirely.

A three-dimensional account of post-quantum migration cost. Section 5 follows a chain of consequences: KEM parameter sizes determine datagram sizes, datagram sizes meet path MTUs, and a datagram that exceeds the IPv6 minimum MTU has no guaranteed path. An in-band ML-KEM-768 initiation is 1300 B against a usable payload of 1232 B. The cost of migration is not only computational: it appears simultaneously in packet size, in path reachability, and in the strength of the binding assumption the KEM must satisfy.

Reported failure, and numbers that cannot drift. Of 12 ProVerif properties, 3 did not terminate within the budget this project set itself, and two of them were decided by Tamarin in seconds. Both facts are in the result table alongside the proofs. So is the mechanism that keeps them honest: the tables, the figures and every quantity quoted in this text are generated from `results/results.json` at build time (Figure 1), so a sentence in this paper cannot survive a run that contradicts it.

1.2. Scope

This paper concerns the handshake. The data plane, the cookie-based rate-limiting mechanism, and endpoint roaming

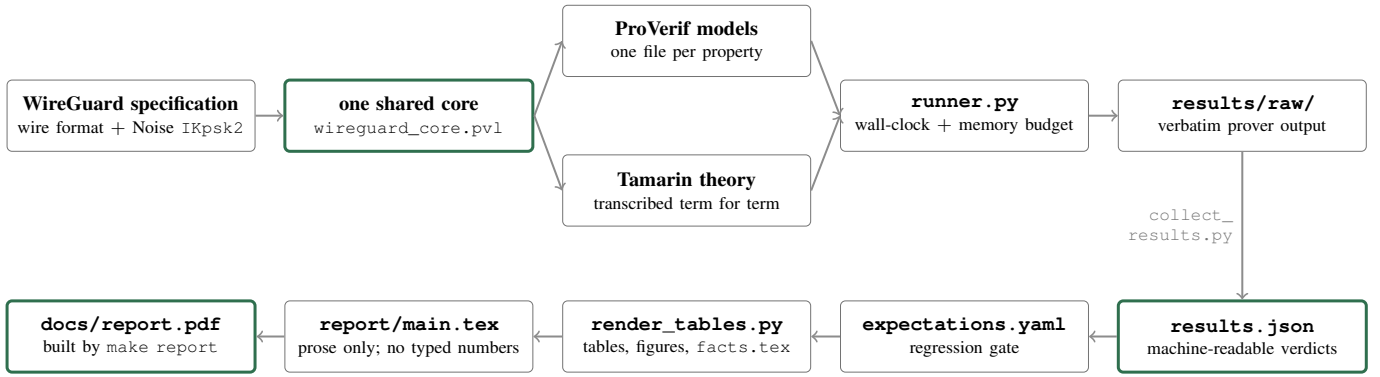


Fig. 1. How one protocol description reaches two provers and one report. The two shaded stages are the load-bearing ones: a single shared core that both prover front-ends include, and a single machine-readable result file that every table, figure and quoted number in this paper is generated from. No number in this document is typed by hand.

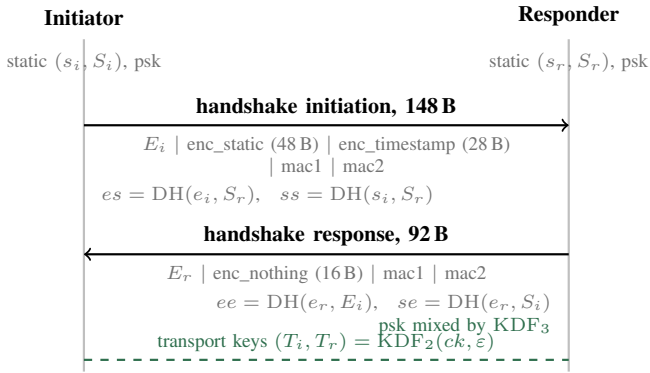


Fig. 2. The two handshake messages, their sizes on the wire, and the Diffie-Hellman terms each contributes to the key schedule. Sizes are computed from the field layout of Section 3, not transcribed.

are network-layer behaviours with no counterpart in an abstract Noise model; Section 3 states for each wire-format field whether it is represented in the models and, where it is not, why. Reproduction instructions and the resource profile of every result accompany the artifact.

2. Background and Related Work

2.1. What the WireGuard handshake computes

WireGuard’s handshake is Noise IKpsk2, instantiated with X25519, ChaCha20-Poly1305 and BLAKE2s. The initiator knows the responder’s static public value in advance—this is what makes the pattern IK rather than XX—and the exchange completes in two messages, an initiation of 148 B and a response of 92 B (Figure 2).

Four Diffie-Hellman terms enter the key schedule, and every property in this paper turns on which of them an attacker can compute:

- $es = \text{DH}(e_i, S_r)$, initiator ephemeral against responder static, binding message 1 to its intended recipient;
- $ss = \text{DH}(s_i, S_r)$, static-static, which gives IK its first-message authentication;

- $ee = \text{DH}(e_r, E_i)$, ephemeral-ephemeral, the term that carries forward secrecy;
- $se = \text{DH}(e_r, S_i)$, responder ephemeral against initiator static, which authenticates the initiator to the responder in message 2.

A pre-shared key, when configured, is mixed in by KDF_3 during message 2, yielding a chaining key, a value τ folded into the transcript hash, and an AEAD key. Because τ enters the transcript, a mismatched pre-shared key makes the authenticated decryption in message 2 fail rather than producing a silently divergent session.

Beyond the Noise pattern. WireGuard adds a TAI64N timestamp, session indices, and two MAC fields. The first, `mac1`, is keyed by a hash of the recipient’s static public value; it demonstrates knowledge of that value and suppresses unsolicited traffic, but is not a cryptographic authenticator. The second, `mac2`, carries a cookie under load. Both are consequential for the correspondence of Section 3 and for what the symbolic model can and cannot say.

2.2. Symbolic verification, and why two provers

Both tools used here work in the Dolev-Yao model [5]: the attacker controls the network completely and the cryptographic primitives are perfect.

ProVerif. ProVerif [6] translates an applied pi-calculus process into Horn clauses and saturates them. It is fully automatic and fast, handles an unbounded number of sessions, and over-approximates: a proof is sound, while a reported attack may occasionally be spurious. Saturation either converges or it does not, and when it does not the tool gives no partial answer.

Tamarin. Tamarin [7] works with multiset rewriting and constraint solving, represents mutable state directly, and normalises Diffie-Hellman terms natively. It reasons backwards from the property rather than forwards from the rules, which turns out to matter for exactly the queries ProVerif could not settle.

Stating authentication precisely. Authentication is stated throughout in Lowe’s hierarchy [8]: aliveness, weak agreement, non-injective agreement, injective agreement. Naming the level

TABLE I

VERIFICATION SCOPE, MEASURED FROM THE REPOSITORY RATHER THAN DESCRIBED. “PROPERTIES” COUNTS ROWS OF THE RESULT MATRIX (TABLE IV), NOT QUERIES: A MODEL FILE MAY CARRY SEVERAL.

Component	Files	Lines	Properties
Shared ProVerif core (<code>lib/</code>)	3	349	—
ProVerif query models	11	302	12
Tamarin theory	1	236	6
Wire-format layout (<code>wire.py</code>)	1	201	25 fields

explicitly matters here, because the key-compromise result of Section 4 is precisely a case of two adjacent levels separating.

2.3. Related work

Prior analyses of WireGuard. Donenfeld and Milner [3] give a symbolic analysis in Tamarin covering the handshake and its state machine. Lipp, Blanchet and Bhargavan [4] give a mechanised computational proof in CryptoVerif, which yields concrete bounds this work cannot. Our contribution relative to both is not a stronger result on the same property, but the deliberate pairing of two symbolic tools on one transcription, and the extension of the analysis past the cryptography into path reachability.

Post-quantum WireGuard. Hülsing et al. [9] redesign the handshake around KEMs directly, and Rosenpass [10] instead runs a post-quantum key agreement on a separate channel and injects the result into WireGuard’s pre-shared key slot. Section 5 does not propose a third design; it supplies the arithmetic that explains why the second shape exists.

KEM binding. Cremers, Dax and Medinger [11] systematise binding properties for KEMs and provide the vocabulary in which the result of Section 4 is stated. The property that separates our two KEM models is MAL-BIND-K-PK.

3. How the Models Are Built and Kept Honest

Table I states what the artifact contains. The figures in it are counted from the files themselves at build time, so the description cannot drift away from the thing it describes.

3.1. One transcription, two provers

The project’s central methodological commitment is that any difference between the two provers’ outcomes should be attributable to the provers, not to the modelling. The handshake is therefore written once, in `lib/wireguard_core.pvl`, and every ProVerif query file includes that library rather than restating the protocol. The Tamarin model is transcribed term for term against it, and Section 6 records the two places where the transcriptions could not be made identical, together with the reason in each case.

3.2. Primitives, and one abstraction that decided the project

Diffie-Hellman with two symbols. Diffie-Hellman is modelled with two symbols rather than an iterated exponentiation:

$$pk(x), \quad dh(P, y), \quad dh(pk(x), y) = dh(pk(y), x).$$

phase 0 — sessions run	phase 1 — after the fact
Dolev–Yao network control. Every ephemeral public value travels in clear, so all of it can be recorded.	Static scalars released and a discrete-logarithm oracle exposed: every recorded public value inverts.
<i>which properties depend on this staging</i>	
P3 forward secrecy, statics released	proved
P4 post-quantum FS, psk secret	proved
P4c the same model, psk published	attack found
S6a/S6b KCI, responder static leaked	the gap

Fig. 3. Compromise is staged, never built in. Phase 0 is the classical Dolev-Yao attacker; phase 1 releases the static scalars and a discrete-logarithm oracle, strictly after every phase-0 session. The key-compromise queries sit outside this staging, because there the compromise is part of the scenario rather than a sequel to it.

Two consequences matter. Because pk is the only constructor of the group type, every group element the attacker can supply is g^t for some term t it can build—faithful for a prime-order group, where every X25519 output is of that form. Because dh returns key material rather than a group element, a Diffie-Hellman result cannot re-enter as a group element, so term depth is bounded by construction.

Why the obvious formulation fails. This was not a stylistic choice. The natural formulation $exp(G, exponent) : G$ lets the attacker build towers $exp(exp(exp(g, a), b), c)$ that the single commuting equation does not normalise. With that formulation, saturation on the secrecy query exhausted the memory budget after roughly 1.2×10^4 clauses and never terminated; with the two-symbol formulation the same query is proved well inside the budget (Table IV). The episode is recorded because it decides whether a symbolic model is usable at all, and it is rarely written down.

A flat transcript hash. Noise updates $h \leftarrow \text{HASH}(h \parallel f)$ after each field. Since h is never transmitted and serves only as AEAD associated data, its sole requirement is that it bind the transcript prefix injectively, so we model h after step i as a distinct collision-resistant function of the entire prefix. Both formulations bind the same data; the flat one keeps term depth constant. AEAD, the HKDF chain and the keyed MAC are standard perfect primitives, with one function symbol per HKDF output position.

3.3. The attacker, and when it gains each capability

The attacker is Dolev-Yao. Compromise is never a built-in capability: each class of secret is revealed by an explicit action, so every query states which compromises it tolerates, and a proof cannot silently depend on an assumption nobody wrote down.

Staging a quantum transition. Post-quantum capability is expressed with ProVerif’s phase mechanism (Figure 3). Phase 0 is the classical attacker. In phase 1 the attacker gains a discrete-logarithm oracle, private in phase 0 and exposed only by a phase-1 process. Since every ephemeral public value travels in

TABLE II

HANDSHAKE INITIATION, 148 B. OFFSETS ARE DERIVED FROM LENGTHS; “MODEL TERM” NAMES THE CORRESPONDING TERM IN THE SHARED PROVERIF CORE, AND A DASH MARKS A FIELD DELIBERATELY OUTSIDE THE MODELS.

Off	Len	Field	Model term
0	1	message_type	—
1	3	reserved	—
4	4	sender_index	—
8	32	unencrypted_ephemeral	$E_i = pk(e_i)$
40	48	encrypted_static	$aead(k_1, 0, S_i, h_1)$
88	28	encrypted_timestamp	$aead(k_2, 0, ts, h_2)$
116	16	mac1	$mac(H(L S_r), \cdot)$
132	16	mac2	—

TABLE III

HANDSHAKE RESPONSE, 92 B. NOTE THAT MAC1 HERE IS KEYED BY THE *initiator*’s STATIC PUBLIC VALUE, WHICH IS WHAT MAKES IDENTITY HIDING CONDITIONAL.

Off	Len	Field	Model term
0	1	message_type	—
1	3	reserved	—
4	4	sender_index	—
8	4	receiver_index	—
12	32	unencrypted_ephemeral	$E_r = pk(e_r)$
44	16	encrypted_nothing	$aead(k_3, 0, \epsilon, h_5)$
60	16	mac1	$mac(H(L S_i), \cdot)$
76	16	mac2	—

clear, an attacker that recorded phase-0 traffic can invert all of it once phase 1 begins. This is the standard formalisation of harvest-now-decrypt-later, and it is what makes post-quantum forward secrecy a property rather than a slogan.

Compromise as part of the scenario. Key-compromise impersonation needs the opposite arrangement: the responder’s static key is in the attacker’s hands *while* the session runs, not afterwards. That is an ordering constraint over individual events rather than a global staging, which is why those queries are posed to Tamarin with action facts and why the two provers do not divide this work symmetrically.

3.4. From wire format to model terms

A formal model earns its authority from the fidelity of its correspondence to the artifact it claims to describe. We therefore maintain a field-by-field mapping from the wire format to terms in the models, and make it machine-checkable rather than a promise.

Offsets are derived, never written. `tools/wire.py` is the single source of truth for the layout. Offsets are accumulated from the field lengths, so a corrected length cannot leave a stale offset behind it, and the computed totals are checked against the specification figures of 148, 92, 64 and 16 B. Tables II and III are rendered from that same structure, which is also what `scripts/verify_wire_table.py` cross-checks the models against. Of 25 fields across the four message types, 7 carry a model term and 18 deliberately do not.

What is not modelled, and why. Four field classes are deliberately absent, and stating the reason for each is part of the correspondence rather than an apology for it. *Message type and reserved bytes* serve demultiplexing; the models use distinct channels instead. *Session indices* are chosen freely by the sender and carry no security burden, but they are also what makes endpoint roaming possible—a WireGuard engineering advantage with no counterpart in an abstract Noise model. *mac2 and the cookie mechanism* are load-shedding devices, and the Dolev-Yao attacker has no notion of computational cost, so a symbolic model cannot express the property they provide. *The data-plane counter and its anti-replay window* lie outside the handshake entirely.

An argued property, not a verified one. The cookie mechanism deserves the distinction made explicitly. `mac1` is keyed by $HASH(LABEL_MAC1 || S_r)$, so an attacker ignorant of the responder’s static public value cannot produce a message that survives the first check, and cannot force a Diffie-Hellman operation. That is a genuine property, and it is genuinely outside the model. We argue it here rather than claim the models establish it.

Conditional identity hiding. One field deserves separate mention. As Table III records, the `mac1` of the *response* is keyed by the *initiator*’s static public value. An observer holding a candidate public key can therefore test whether a given response is addressed to its holder. WireGuard’s identity hiding is accordingly conditional: it protects the initiator’s identity from an observer who does not already hold the candidate keys, and not from one who does. The distinction is visible only because `mac1` was modelled explicitly rather than abstracted away.

3.5. Budgets, expectations, and generated numbers

Every run is bounded. Every invocation carries an explicit address-space cap and wall-clock timeout—2.8 GB and 300 s on a single core, a budget chosen to match what remains available on an 8 GB laptop with an editor open. The cap matters more than its value: without one, a runaway query consumes the machine and produces a run that is slow, irreproducible and hostile to anything else running. With one, a query either finishes within budget or fails deterministically, which is a reportable outcome.

Verification as a regression suite. `expectations.yaml` declares the expected verdict for each of 18 queries, and `make verify` fails on any mismatch. Outcomes that are attacks are expectations too: the control model of Section 4 is *supposed* to be refuted, and a run in which it succeeded would indicate that the model had drifted, not that the protocol had improved.

No number in this paper is typed. Result tables, figures and every quantity this text quotes are rendered from `results/results.json`, `tools/wire.py` and `tools/pqcost.py` when the document is built. An earlier revision of this report was internally consistent and externally wrong—the tables were generated and correct while the prose quoted timings from a run nobody could identify. Generating

the tables was not enough, because prose is not data; the numbers in the sentences are now macros too.

4. Verification Results

Table IV is the complete result matrix, rendered from `results/results.json`. Every cell is populated: proved, refuted, inconclusive, or non-terminating with a recorded cause and the budget it consumed. Of 18 properties, 11 are proved, 3 are refuted, 1 is inconclusive and 3 did not terminate.

4.1. Baseline secrecy and authentication

Secrecy. Session-key secrecy (P1) and forward secrecy (P3) are proved with an unbounded number of concurrent sessions. Forward secrecy is stated by releasing both static scalars in phase 1, strictly after every phase-0 session; the ephemeral-ephemeral term ee is what survives.

Authentication, level by level. Non-injective agreement of the responder with the initiator (P5b) is proved, and so is injective agreement (P5c). Injectivity is simultaneously the replay-resistance statement: a replayed initiation would produce two responder completions against a single initiator run, which is exactly what the injective query forbids, and the TAI64N timestamp is what prevents it.

The direction that resists. Agreement in the opposite direction (P5a) did not terminate. Three formulations were attempted—agreement on both static values and both transport keys, an existentially weakened aliveness statement, and the same with `redundantHypElim` enabled—and each exhausted the budget. The direction that resists is the one whose proof must reason through the authenticated decryption of message 2 under a psk -derived key, the deepest point in the key schedule.

4.2. What the pre-shared key actually contributes

The claim that `IKpsk2`'s pre-shared key provides post-quantum protection is common and rarely tested. Table V tests it. Both models are the same file up to one line: whether the pre-shared key is published. Both face an attacker that, in phase 1, can invert every Curve25519 public value it recorded—which is every ephemeral, since they travel in clear, and both statics.

With the key public. Every input to the key schedule is recoverable and `ProVerif` returns a derivation reconstructing both transport keys; the derivation is archived under `results/traces/`.

With the key secret. The same attacker is defeated: KDF_3 mixes an independent secret into the chaining key, and no amount of discrete-logarithm capability recovers it. Because the two models differ in exactly one variable, the difference in outcome is attributable to that variable and to nothing else. This is the sense in which the pre-shared key is doing post-quantum work, and the control is what makes the statement more than an assertion.

1. Forge message 1

needs $es = DH(e_i, S_r)$ and $ss = DH(s_i, S_r)$
both computable from the leaked responder scalar

S6a falsified — the responder commits key material to a session it attributes to an honest initiator

the gap se holds open

2. Complete the session

additionally needs $se = DH(e_r, S_i)$
requires the initiator's static scalar, which is not compromised

S6b verified — no transport record is ever accepted on that session

Fig. 4. With the responder's static key compromised, two adjacent authentication levels separate. The static-ephemeral term se is the whole of the difference, and the property lives in the gap between them.

4.3. Where KCI resistance resides

Key-compromise impersonation asks whether compromising a party's own long-term key lets an attacker impersonate *others* to it. We asked it at two levels, and the levels separate (Figure 4).

The responder does commit. Producing a well-formed message 1 in the name of an honest initiator requires $es = DH(e_i, S_r)$ and $ss = DH(s_i, S_r)$. Both are computable from the leaked responder scalar, so an attacker can drive the responder to commit transport keys for a session it attributes to that initiator. Tamarin refutes the commitment-level lemma (S6a) and returns the trace.

The responder never completes. Deriving those transport keys additionally requires $se = DH(e_r, S_i)$, and that needs either the responder's ephemeral scalar or the initiator's static scalar. The attacker has neither. Tamarin verifies the completion-level lemma (S6b): the responder never accepts a transport record on such a session.

Why the distinction is operational. WireGuard's KCI resistance depends on treating a session as established only once authenticated traffic has arrived, not once the handshake response has been sent. A model with a single "session established" event would report one verdict where there are two, and would locate the property in the wrong place.

4.4. KEM-derived pre-shared keys

If the pre-shared key is to come from a KEM rather than from manual configuration—the construction Rosenpass [10] uses—the question is what that KEM must bind. We modelled two: one whose shared secret depends on the encapsulation key, and one whose shared secret depends only on the encapsulation randomness, together with the re-encapsulation capability that absence of binding confers.

Binding is the whole of it. With binding, secrecy of the established pre-shared key is proved (P12a). Without it, the

TABLE IV

VERIFICATION OUTCOMES, THE COMPLETE MATRIX. PROVERIF 2.05 AND TAMARIN 1.12.0, SINGLE CORE, 2.8 GB ADDRESS-SPACE CAP, 300 S WALL-CLOCK BUDGET. † MARKS A BUDGET CONSUMED WITHOUT REACHING A VERDICT, WHICH IS NOT A TIME-TO-VERDICT AND MUST NOT BE READ AS ONE; *n/r* MARKS A RUN THAT PREDATES PER-QUERY COST RECORDING. EVERY CELL IS RENDERED FROM `RESULTS/RESULTS.JSON`.

ID	Property	Prover	Cost	Outcome
P1	Session-key secrecy	ProVerif	19 s	proved
P3	Forward secrecy	ProVerif	20 s	proved
P4	Post-quantum forward secrecy (psk)	ProVerif	20 s	proved
P4c	same, psk absent (control)	ProVerif	20 s	attack found
P5a	Agreement, responder to initiator	ProVerif	300 s†	non-terminating
P5b	Agreement, initiator to responder	ProVerif	23 s	proved
P5c	Injective agreement / replay (P9)	ProVerif	24 s	proved
P6a	KCI, commitment level	ProVerif	300 s†	non-terminating
P6b	KCI, completion level	ProVerif	300 s†	non-terminating
P12a	KEM-derived psk secrecy, K-PK binding	ProVerif	<1 s	proved
P12b	same, binding absent (control)	ProVerif	<1 s	attack found
P12c	KEM-derived psk, responder-view soundness	ProVerif	<1 s	inconclusive
S0	Model admits an honest run	Tamarin	3 s	proved
S1	Session-key secrecy	Tamarin	2 s	proved
S3	Forward secrecy	Tamarin	2 s	proved
S5	Agreement, no compromise	Tamarin	2 s	proved
S6a	KCI, commitment level	Tamarin	3 s	attack found
S6b	KCI, completion level	Tamarin	2 s	proved

TABLE V

THE POST-QUANTUM CONTROL EXPERIMENT: TWO MODELS, ONE VARIABLE. BOTH ARMS FACE THE SAME ATTACKER, WHICH GAINS A DISCRETE-LOGARITHM ORACLE IN PHASE 1. THE VERDICT COLUMN IS READ FROM THE SAME RESULT FILE AS TABLE IV, SO THE EXPERIMENT CANNOT BE REPORTED AS HAVING SUCCEEDED IN A RUN WHERE IT DID NOT.

Model	psk	Transport-key secrecy
22_pq_forward_secrecy.pv	secret	proved
23_pq_fs_nopsk.pv	public	attack found

attacker redirects an observed ciphertext to an encapsulation key of its own and recovers the pre-shared key outright (P12b). The failure is not a weakness in any primitive: confidentiality and integrity of the KEM are untouched, and only the binding is missing. In the vocabulary of Cremers, Dax and Medinger [11] the separating property is MAL-BIND-K-PK, which ML-KEM satisfies.

One query we could not settle. A third query, asking whether the responder’s view of the established key is sound, returned inconclusive (P12c): a bare encapsulation authenticates nobody, and the honest statement is disjunctive in a way ProVerif’s approximation could not decide. We report it as inconclusive rather than restate it until it passed. An undecided query is a result; a restated one is not.

5. The Cost of Post-Quantum Migration

The analysis in this section requires no computation and no measurement. Every figure follows arithmetically from the wire-format layout of Section 3 and the published parameter sizes of ML-KEM [12]. That makes it the most robust part of the paper: it does not depend on the machine it was produced on, and it cannot be disturbed by a prover’s resource limits.

TABLE VI

IN-BAND KEM SUBSTITUTION, SIZES IN BYTES. THE SUBSTITUTION MODELLED IS THE CHEAPEST ONE POSSIBLE, SO THESE ARE A LOWER BOUND ON ANY IN-BAND DESIGN.

KEM	<i>ek</i>	<i>ct</i>	initiation	response
X25519 (baseline)	32	32	148	92
ML-KEM-512	800	768	916 (×6.2)	828 (×9.0)
ML-KEM-768	1184	1088	1300 (×8.8)	1148 (×12.5)
ML-KEM-1024	1568	1568	1684 (×11.4)	1628 (×17.7)

TABLE VII

ML-KEM-768 INITIATION AGAINST 5 REPRESENTATIVE PATH MTUS. ONLY THE FIRST ROW IS A GUARANTEE; THE OTHERS ARE COMMON CASES.

Path	MTU	usable UDP	ML-KEM-768 initiation
IPv6 minimum MTU	1280	1232	exceeds by 68 B
Ethernet, IPv4	1500	1472	fits, 172 B spare
Ethernet, IPv6	1500	1452	fits, 152 B spare
PPPoE, IPv4	1492	1464	fits, 164 B spare
Tunnel stacking, IPv4	1400	1372	fits, 72 B spare

5.1. Packet growth

We model the cheapest possible in-band substitution: the initiator’s 32-byte ephemeral public value becomes an encapsulation key, the responder’s becomes the corresponding ciphertext, and every other field keeps its length. Because this is the minimal construction, the resulting sizes (Table VI) are a *lower bound* on any in-band design, which is what makes the conclusion below robust rather than contingent on a particular protocol sketch. An initiation grows by a factor of 8.8 at ML-KEM-768 and 11.4 at ML-KEM-1024.

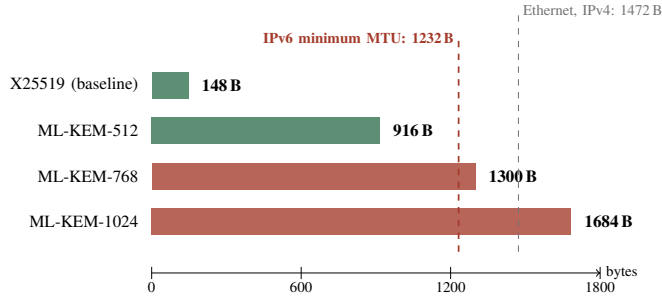


Fig. 5. Initiation size against the usable UDP payload of each path. The ML-KEM-768 bar crosses the IPv6 minimum-MTU rule; that crossing is the whole of the argument in this section.

5.2. Path reachability

RFC 8200 [13] requires every IPv6 link to carry a 1280-byte datagram. After an IPv6 header of 40 B and a UDP header of 8 B, the usable payload is 1232 B. An in-band ML-KEM-768 initiation is 1300 B. It exceeds that payload by 68 B, and a datagram that does not fit the IPv6 minimum MTU has no guaranteed path (Figure 5, Table VII). Path MTU discovery for datagram transports [14] can find a larger path when one exists, but it cannot manufacture one where it does not.

The chain, stated plainly. Each link below is individually unremarkable, and the conclusion is not:

KEM parameter sizes → datagram size → path
 MTU → fragmentation or PMTUD failure → the
 handshake is unreachable on some paths → the
 architecture must change.

This is why post-quantum designs for WireGuard either restructure the handshake around KEMs from the start [9] or perform the key agreement over a separate channel and inject the result into the pre-shared key slot. Rosenpass [10] takes the second route, and the analysis above supplies the reason it is necessary rather than merely convenient.

What migration costs beyond bytes. There is a real tension worth naming. WireGuard’s single-packet, stateless handshake is itself an engineering asset: it needs no fragment reassembly, holds no state before authentication, and is what makes the cookie mechanism possible. Post-quantum migration threatens precisely that asset. ML-KEM-512 would fit every path considered at 916 B, but at a security level most deployments will not choose; ML-KEM-1024, at 1684 B, fits none of them.

5.3. Three dimensions, one conclusion

Combining this section with Section 4:

- *Packet size.* An in-band initiation grows by a factor of 8.8 at ML-KEM-768 and 11.4 at ML-KEM-1024.
- *Path reachability.* Above ML-KEM-512, in-band substitution loses the guarantee of a usable path.
- *Assumption strength.* A KEM-derived pre-shared key preserves the properties of a configured one only if the KEM binds its shared secret to the encapsulation key;

TABLE VIII
 THE TWO PROVERS ON ONE TRANSCRIPTION. “WALL-CLOCK SPENT UNDECIDED” IS BUDGET CONSUMED WITHOUT PRODUCING A VERDICT; IT IS A COST, NOT A RESULT.

	ProVerif 2.05	Tamarin 1.12.0
Properties posed	12	6
Proved	6	5
Attack found	2	1
Inconclusive	1	0
Non-terminating	3	0
Wall-clock to a verdict (s)	<1–24	2–3
Wall-clock spent undecided (s)	900	—

ProVerif

P1	Session-key secrecy	19 s
P3	Forward secrecy	20 s
P4	Post-quantum forward secrecy (...)	20 s
P4c	same, psk absent (control)	20 s
P5a	Agreement, responder to initia...	300 s [†]
P5b	Agreement, initiator to respon...	23 s
P5c	Injective agreement / replay (...)	24 s
P6a	KCI, commitment level	300 s [†]
P6b	KCI, completion level	300 s [†]
P12a	KEM-derived psk secrecy, K-PK ...	<1 s
P12b	same, binding absent (control)	<1 s
P12c	KEM-derived psk, responder-vie...	<1 s

Tamarin

S0	Model admits an honest run	3 s
S1	Session-key secrecy	2 s
S3	Forward secrecy	2 s
S5	Agreement, no compromise	2 s
S6a	KCI, commitment level	3 s
S6b	KCI, completion level	2 s

0 150 300
seconds

Fig. 6. Wall-clock cost of every property, grouped by prover. The bars marked [†] reached the budget without reaching a verdict; two of the three are the properties Tamarin settles in the group below.

without that binding the pre-shared key is recoverable while every primitive remains unbroken (P12a/P12b).

The cost of post-quantum migration is not merely computational. It appears along all three dimensions at once, and a design that addresses only one of them has not addressed the problem.

6. Prover Comparison and Limitations

Because the protocol was transcribed once and handed to both provers, the differences reported here are attributable to the provers. Table VIII aggregates the result matrix by tool.

6.1. Where the transcriptions diverge

Two differences were unavoidable, and both are consequences of how the provers handle Diffie-Hellman.

Term construction. ProVerif required the two-symbol pk/dh encoding of Section 3 to bound term depth. Tamarin normalises Diffie-Hellman terms natively and uses its built-in theory directly. The two formulations agree on the equational theory; they differ in what the attacker can syntactically construct, and only one of the provers needs the restriction.

Expressing compromise. Compromise is expressed with ProVerif’s phase mechanism and with action facts in Tamarin. Phases impose a global ordering, which is exactly what harvest-now-decrypt-later needs and is why the post-quantum experiment lives on the ProVerif side. Action facts let a lemma quantify over the relative order of individual compromise and session events, which is what the KCI lemmas need.

6.2. The resource profile, and what it is evidence for

The contrast is sharp and one-directional. ProVerif proved what it could prove quickly—<1–24 s across the properties it decided—and failed completely on the rest, exhausting the budget rather than slowing down. Tamarin decided every lemma we posed it, including both KCI levels, in 2–3 s and 7–18 constraint-solving steps (Figure 6).

An identifiable cause. The 3 ProVerif properties left undecided are agreement in the responder-to-initiator direction and both KCI levels. The KCI failures have a diagnosis. Releasing the responder’s static scalar unlocks every responder-side Diffie-Hellman term for the attacker, and the resulting clause set diverges; bounding the initiator to a single session reduced memory pressure but not the divergence. Tamarin’s constraint solving is not sensitive in the same way, because it reasons backwards from the property rather than saturating forwards from the rules.

A practical rule. The division of labour that emerged is not the textbook one, and it is worth stating in the form this project actually found it. Reach for ProVerif when the property is secrecy or a correspondence over honest parties, when many variants of one model must be run, or when a global staging of attacker capability—such as a quantum transition—is the natural formulation; it is fast and fully automatic in that regime. Reach for Tamarin when a long-term secret is compromised as part of the scenario rather than after it, when the property depends on the relative ordering of specific events, or when explicit mutable state is required. The KCI pair of Section 4 is all three at once, and it is the single clearest instance in this work.

What this is not. This is a resource observation from one protocol, not a general claim about the tools. It is recorded because comparisons of this kind are rarely published with the failures included, and the failures are the informative part.

6.3. Limitations

The symbolic abstraction. Both models assume a Dolev-Yao attacker and perfect primitives. They say nothing about side channels, implementation defects, or concrete security parameters. The Diffie-Hellman abstraction treats every 32-byte value as g^t for some t , which excludes small-subgroup and invalid-curve behaviour and the all-zero output check.

The transcript hash. Modelling h after step i as a flat collision-resistant function of the prefix binds the same data as the iterated formulation, but it assumes the hash chain is collision-free as a whole rather than deriving that from the

properties of a single compression function. A collision attack on BLAKE2s would be invisible to this model, as it would be to any symbolic one.

Non-terminating queries prove nothing. Of 18 properties, 3 were left undecided. A non-terminating query is not a proof of absence: it says the prover could not settle the question within the budget, and nothing about the protocol. Two of the three were subsequently decided by Tamarin; the third, agreement in the responder-to-initiator direction, remains open in this artifact. The KCI queries were additionally attempted on the ProVerif side with the initiator bounded to one and two sessions; those runs would have established the property for that number of sessions only, and are reported as non-terminating in any case.

Verification, not measurement. This artifact verifies the handshake. It does not measure it. The network laboratory accompanying the repository—a multi-node network namespace topology with traffic capture and injected wide-area conditions—is provided as a Linux companion and was not exercised for this release, because network namespaces are a Linux kernel interface with no macOS equivalent and the development machine is an Apple Silicon laptop. No latency or throughput figure appears anywhere in this paper, and none should be inferred from it. Section 5 depends on no measurement whatsoever.

Attribution. Where the conclusions here differ from prior work [3], [4], the difference is a modelling assumption rather than a finding, and the assumptions are stated in Section 3 so that such differences can be traced.

7. Conclusion

Transcribing the WireGuard handshake once and giving it to two provers produced three findings a single-tool analysis would not have, and one property of the artifact itself.

The pre-shared key’s contribution became an experiment. Two models differing in one line, one proved and one refuted against an attacker holding a discrete-logarithm oracle. What was previously an assertion about IKpsk2 is now a control with a recorded verdict on each arm (Table V).

KCI resistance turned out to have a location. It is not a property of the handshake as a whole but of the interval between committing transport keys and accepting traffic, and the static-ephemeral term se is what holds that interval open. Finding this required asking the question at two levels, and answering it required the prover that ProVerif’s saturation had defeated.

Post-quantum cost is not one number. Following KEM parameter sizes through to path MTU shows an in-band ML-KEM-768 handshake exceeding the IPv6 minimum by 68 B. The cost appears as packet size, path reachability, and assumption strength simultaneously, and a design that addresses one of the three has not addressed the problem.

The disagreements are the informative part. The 3 properties nobody could decide sit in Table IV alongside the rest. A result matrix in which every cell is green says less about a pair of

provers than one in which the disagreements are visible. That this remains true of every future run is not a promise but a build step: the tables, figures and quoted numbers in this paper are regenerated from the provers' own output each time the document is compiled.

References

- [1] J. A. Donenfeld, "WireGuard: Next generation kernel network tunnel," in *Network and Distributed System Security Symposium (NDSS)*, 2017.
- [2] T. Perrin, "The noise protocol framework, revision 34," 2018, <https://noiseprotocol.org/noise.html>.
- [3] J. A. Donenfeld and K. Milner, "Formal verification of the WireGuard protocol," University of Oxford, Tech. Rep., 2018.
- [4] B. Lipp, B. Blanchet, and K. Bhargavan, "A mechanised cryptographic proof of the WireGuard virtual private network protocol," in *IEEE European Symposium on Security and Privacy (EuroS&P)*, 2019.
- [5] D. Dolev and A. C. Yao, "On the security of public key protocols," *IEEE Transactions on Information Theory*, vol. 29, no. 2, pp. 198–208, 1983.
- [6] B. Blanchet, "Modeling and verifying security protocols with the applied pi calculus and ProVerif," *Foundations and Trends in Privacy and Security*, vol. 1, no. 1–2, pp. 1–135, 2016.
- [7] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN prover for the symbolic analysis of security protocols," in *Computer Aided Verification (CAV)*, 2013.
- [8] G. Lowe, "A hierarchy of authentication specifications," in *IEEE Computer Security Foundations Workshop (CSFW)*, 1997.
- [9] A. Hülsing, K.-C. Ning, P. Schwabe, F. Weber, and P. R. Zimmermann, "Post-quantum WireGuard," in *IEEE Symposium on Security and Privacy (S&P)*, 2021, pp. 304–321.
- [10] K. Varner, B. Lipp, W. Zaeske, and L. Schmidt, "Rosenpass: A formally verified post-quantum secure key exchange for WireGuard," 2023, <https://rosenpass.eu>.
- [11] C. Cremers, A. Dax, and N. Medinger, "Keeping up with the KEMs: Stronger security notions for KEMs and automated analysis of KEM-based protocols," in *ACM Conference on Computer and Communications Security (CCS)*, 2024.
- [12] National Institute of Standards and Technology, "FIPS 203: Module-lattice-based key-encapsulation mechanism standard," 2024.
- [13] S. Deering and R. Hinden, "RFC 8200: Internet protocol, version 6 (IPv6) specification," 2017.
- [14] G. Fairhurst, T. Jones, M. Tüxen, I. Rüngeler, and T. Völker, "RFC 8899: Packetization layer path MTU discovery for datagram transports," 2020.